Alumno: Luis Rodriguez Palacios

Modulo: Entornos de Desarrollo

Ciclo: Grado superior Desarrollo de Aplicaciones Multiplataforma

Fecha: 23/01/2026

Profesor: Cesar Brera Marquez

Titulo: JUnit

Contenido

Junit 3

Escribe una batería de pruebas para el siguiente método de la clase Fecha que se expone a continuación. El método recibe un número entero y devuelve un String con un formato de fecha que dependerá del valor del número entero. Si el número recibido es diferente de 1, 2, y 3 se devolverá “ERROR”. 3

Escribe una clase de pruebas para el método calculo de la calse Factorial. En el método se comprueba si el número es menor que 0, en este caso se lanza la excepción IllegalArgumentException con el mensaje “El numero no puede ser negativo o 0”. Si el valor del factorial calculado es menor que 0 es que ha ocurrido un error de desbordamiento, en este caso se lanza la excepción ArithmeticException y se lanza el mensaje “Overflow”. La clase a probar es la siguiente: 5

Analiza el código de la siguiente clase y desarrolla una batería de pruebas en JUnit para cada uno de los métodos y constructores. 7

Junit

Escribe una batería de pruebas para el siguiente método de la clase Fecha que se expone a continuación. El método recibe un número entero y devuelve un String con un formato de fecha que dependerá del valor del número entero. Si el número recibido es diferente de 1, 2, y 3 se devolverá “ERROR”.

JUnit > src > main > java > com > iescomercio > Fecha.java > Fecha > hoy

```java
package com.iescomercio;
/*
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */


import java.text.SimpleDateFormat;
import java.util.Date;

/**
 * @author LRP
 */
public class Fecha {

    SimpleDateFormat formato;
    Date hoy;

    public Fecha() {
        hoy = new Date();
    }

    public String devuelveFecha(int tipo) {
        String cad = "";
        if (tipo == 1) {
            formato = new SimpleDateFormat(pattern: "yyyy/MM");
            cad = formato.format(hoy);
        } else if (tipo == 2) {
            formato = new SimpleDateFormat(pattern: "MM/yyyy");
            cad = formato.format(hoy);
        } else if (tipo == 3) {
            formato = new SimpleDateFormat(pattern: "MM/yy");
            cad = formato.format(hoy);
        } else {
            cad = "ERROR";
        }
        return cad;
    }
}

```

Creamos la clase FechaTest.java en la subcarpeta test.

Assessment Reports | Assessment Report | FechaTest.java

JUnit > src > test > java > com > iescomercio > FechaTest.java > ...

```java

package com.iescomercio;
/**
 * @author LRP
 */

import static org.junit.Assert.assertEquals;
import org.junit.Test;

public class FechaTest {

    // @Test: Comprueba el formato "tipo 1" (yyyy/MM). Verifica que devuelve exactamente "2026/01"
    // y que la longitud del String es 7 caracteres.
    @Test
    public void testTipo1() {
        Fecha fecha = new Fecha();
        String resultado = fecha.devuelveFecha(tipo: 1);
        //Formato esperado "yyyy/MM"
        String esperado = "2026/01";
        assertEquals(esperado, resultado);
        assertEquals(7, resultado.length());
    }

    // @Test: Comprueba el formato "tipo 2" (MM/YYYY). Verifica que devuelve exactamente "01/2026"
    // y que la longitud del String es 7 caracteres.
    @Test
    public void testTipo2() {
        Fecha fecha = new Fecha();
        String resultado = fecha.devuelveFecha(tipo: 2);
        //Formato esperado "MM/YYYY"
        String esperado = "01/2026";
        assertEquals(esperado, resultado);
        assertEquals(7, resultado.length());
    }

    // @Test: Comprueba el formato "tipo 3" (MM/YY). Verifica que devuelve exactamente "01/26"
    // y que la longitud del String es 5 caracteres.
    @Test
    public void testTipo3() {
        Fecha fecha = new Fecha();
        String resultado = fecha.devuelveFecha(tipo: 3);
        //Formato esperado "MM/YY"
        String esperado = "01/26";
        assertEquals(esperado, resultado);
        assertEquals(5, resultado.length());
    }

    // @Test: Comprueba el caso de error cuando el tipo no es válido (por ejemplo 0).
    // Verifica que devuelve exactamente "ERROR" y que la longitud del String es 5 caracteres.
    @Test
    public void testError() {
        Fecha fecha = new Fecha();
        String resultado = fecha.devuelveFecha(tipo: 0);
        //Formato esperado "ERROR"
        String esperado = "ERROR";
        assertEquals(esperado, resultado);
        assertEquals(5, resultado.length());
    }
}
```

Hemos chequeado lo siguiente:
De manera positiva (casos correctos)
* devuelveFecha(1) -> se espera "2026/01" (formato yyyy/MM) y longitud 7
* devuelveFecha(2) -> se espera "01/2026" (formato MM/YYYY) y longitud 7
* devuelveFecha(3) -> se espera "01/26" (formato MM/YY) y longitud 5

De manera negativa / error (errores controlados)
X devuelveFecha(0) -> se espera "ERROR" y longitud 5

Para poder realizar los test de manera metódica y eficiente, utilizamos la ayuda de el método de caja blanca para saber cuales serian los resultados esperados correctos e incorrectos y luego nos ayudaríamos de la caja negra para implementar todos los test que ahí aparecen, tanto los positivos como los de error y comprobamos en la aplicación que se cumplen.

Escribe una clase de pruebas para el método calculo de la calse Factorial. En el método se comprueba si el número es menor que 0, en este caso se lanza la excepción IllegalArgumentException con el mensaje “El numero no puede ser negativo o 0”. Si el valor del factorial calculado es menor que 0 es que ha ocurrido un error de desbordamiento, en este caso se lanza la excepción ArithmeticException y se lanza el mensaje “Overflow”. La clase a probar es la siguiente:

JUnit > src > main > java > com > iescomercio > Factorial.java > Factorial > calculo(int)

```java
package com.iescomercio;
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */


/**
 *
 * @author LRP
 */
public class Factorial {

    public static int calculo(int n) {
        if (n < 0) {
                throw new IllegalArgumentException(s: "El numero no puede ser negativo o 0");
        }

        int fact = 1;
        for (int i = 2; i <= n; i++) {
            fact *= i;
        }

        if (fact < 0) {
            throw new ArithmeticException(s: "Overflow");
        }
        return fact;
    }
}
```

Y según este código hacemos su correspondiente clase para el test, FactorialTest.java

JUnit > src > test > java > com > iescomercio > FactorialTest.java > ...

```java
package com.iescomercio;

/**
 * @author LRP
 */

import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertThrows;

import org.junit.Test;

public class FactorialTest {

    // @Test: Comprueba un caso normal (n positivo). Verifica que el factorial de 5 es 120.
    @Test
    public void testPositivo() {
        Factorial factorial = new Factorial();
        int resultado = factorial.calculo(n: 5);
        int esperado = 120; // resultado esperado 120
        assertEquals(esperado, resultado);
    }

    // @Test: Comprueba que, si se pasa un número negativo, el método lanza IllegalArgumentException
    // y además que el mensaje de la excepción sea exactamente el esperado.
    @Test
    public void testNegativo() {
        Factorial factorial = new Factorial();
        IllegalArgumentException excepcion = assertThrows(expectedThrowable: IllegalArgumentException.class, () -> {
            factorial.calculo(-1);
        });
        String esperado = "El numero no puede ser negativo o 0"; // resultado esperado
        assertEquals(esperado, excepcion.getMessage());
    }

    // @Test: Comprueba un caso especial (n = 0). Verifica que el factorial de 0 devuelve 1.
    @Test
    public void testEspecial() {
        Factorial factorial = new Factorial();
        int resultado = factorial.calculo(n: 0);
        int esperado = 1;
        assertEquals(esperado, resultado);
    }

    // @Test: Comprueba un caso de desbordamiento (overflow). Verifica que para un número grande (17)
    // el método lanza ArithmeticException y que el mensaje de la excepción sea "Overflow".
    @Test
    public void testOverflow() {
        Factorial factorial = new Factorial();

        // Con 17 el resultado intermedio se vuelve negativo y salta tu IF
        ArithmeticException excepcion = assertThrows(expectedThrowable: ArithmeticException.class, () -> {
            factorial.calculo(n: 17);
        });

        assertEquals(expected: "Overflow", excepcion.getMessage());
    }

}
```

Aquí hemos chequeado lo siguiente:
De manera positiva (casos correctos)
* calculo(5) -> se espera 120
* calculo(0) -> se espera 1

De manera negativa / error (errores controlados)
X calculo(-1) -> se espera IllegalArgumentException con mensaje "El numero no puede ser negativo o 0"
X calculo(17) -> se espera ArithmeticException con mensaje "Overflow"

Analiza el código de la siguiente clase y desarrolla una batería de pruebas en JUnit para cada uno de los métodos y constructores.

JUnit > src > main > java > com > iescomercio > TablaEnteros.java > ...

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.iescomercio;
/**
 * @author LRP
 */
import java.util.NoSuchElementException;


public class TablaEnteros {

    private Integer[] tabla;

    public TablaEnteros(Integer[] tabla) {
        if (tabla == null || tabla.length == 0) {
            throw new IllegalArgumentException(s: "No hay elementos");
        }
        this.tabla = tabla;
    }

    public int sumaTabla() {
        int suma = 0;
        for (int i = 0; i < tabla.length; i++) {
            suma += tabla[i];
        }
        return suma;
    }

    public int mayorTabla() {
        int max = Integer.MIN_VALUE;
        for (int i = 0; i < tabla.length; i++) {
            if (tabla[i] > max) {
                max = tabla[i];
            }
        }
        return max;
    }

    public int posicionTabla(int n) {
        for (int i = 0; i < tabla.length; i++) {
            if (tabla[i] == n) {
                return i;
            }
        }
        throw new NoSuchElementException(s: "No existe");
    }
}

```

Este es el código y este seria la clase test TablaEnterosTest.java

JUnit > src > test > java > com > iescomercio > TablaEnterosTest.java > ...

```java
package com.iescomercio;

/**
 * @author LRP
 */

import org.junit.Test;

import static org.junit.Assert.assertNotNull;
import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertThrows;

import java.util.NoSuchElementException;

public class TablaEnterosTest {
```

e - Gobierno de La Rioja -
osDesarrollo\JUnit\src\test\java\com\iescomercio\FactorialTest.java • 4

```java
funciona con un array válido y crea el objeto (no null).

    public void testConstructorPositivo() {
        Integer[] miTabla = { 1, 2, 3 };
        // No debería lanzar ninguna excepción
        TablaEnteros tabla = new TablaEnteros(miTabla);
        assertNotNull(tabla);
    }

    // @Test: Comprueba que sumaTabla() suma correctamente números positivos.
    @Test
    public void testSumaTablaPositivo() {
        Integer[] miTabla = { 1, 2, 3, 4, 5 };
        TablaEnteros tabla = new TablaEnteros(miTabla);
        int resultado = tabla.sumaTabla();
        int esperado = 15; // 1+2+3+4+5=15
        assertEquals(esperado, resultado);
    }

    // @Test: Comprueba que sumaTabla() suma correctamente números negativos.
    @Test
    public void sumaTablaNegativo() {
        Integer[] miTabla = { -1, -2, -3, -4, -5 };
        TablaEnteros tabla = new TablaEnteros(miTabla);
        int resultado = tabla.sumaTabla();
        int esperado = -15; // -1-2-3-4-5=-15
        assertEquals(esperado, resultado);
    }

    // @Test: Comprueba que sumaTabla() devuelve 0 cuando todos los valores son 0.
    @Test
    public void testSumaCero() {
        Integer[] miTabla = { 0, 0, 0, 0, 0, 0 };
        TablaEnteros tabla = new TablaEnteros(miTabla);
        int resultado = tabla.sumaTabla();
        int esperado = 0; // 0+0+0+0+0+0=0
        assertEquals(esperado, resultado);
    }
```

JUnit > src > test > java > com > iescomercio > TablaEnterosTest.java > ...

```java
15  public class TablaEnterosTest {
55
56      // @Test: Comprueba que mayorTabla() devuelve el mayor valor en una tabla de positivos.
57      @Test
58      public void testMayorTablaPositivos() {
59          Integer[] datos = { 1, 40, 20, 5, 10 };
60          TablaEnteros objeto = new TablaEnteros(datos);
61          int resultado = objeto.mayorTabla();
62          int esperado = 40; // El mayor debería ser 40
63          assertEquals(esperado, resultado);
64      }
65
66      // @Test: Comprueba que mayorTabla() devuelve el mayor valor cuando todos son negativos
67      // (es decir, el menos negativo).
68      @Test
69      public void testMayorTablaNegativos() {
70          Integer[] datos = { -10, -5, -20, -100 };
71          TablaEnteros objeto = new TablaEnteros(datos);
72          int resultado = objeto.mayorTabla();
73          int esperado = -5; // El mayor debería ser -5 (el menos negativo)
74          assertEquals(esperado, resultado);
75      }
76
77      // @Test: Comprueba que mayorTabla() funciona si el array tiene un único elemento.
78      @Test
79      public void testMayorTablaUnSoloElemento() {
80          Integer[] datos = { 7 };
81          TablaEnteros objeto = new TablaEnteros(datos);
82          int resultado = objeto.mayorTabla();
83          int esperado = 7; // El mayor debería ser 7
84          assertEquals(esperado, resultado);
85      }
86
87      // @Test: Comprueba que posicionTabla(n) devuelve el índice correcto cuando el elemento existe.
88      @Test
89      public void testPosicionEncontrada() {
90          Integer[] datos = { 10, 20, 30, 40 };
91          TablaEnteros objeto = new TablaEnteros(datos);
92          int resultado = objeto.posicionTabla(n: 30);
93          int esperado = 2; // Buscamos el 30, que está en el índice 2
94          assertEquals(esperado, resultado);
95      }
96
97      // @Test: Comprueba que posicionTabla(n) devuelve 0 si el elemento buscado está en la primera posición.
98      @Test
99      public void testPosicionPrimera() {
100         Integer[] datos = { 5, 10, 15 };
101         TablaEnteros objeto = new TablaEnteros(datos);
102         int resultado = objeto.posicionTabla(n: 5);
103         int esperado = 0; // Verificamos que encuentra el primer elemento (índice 0)
104         assertEquals(esperado, resultado);
105     }
106
107     // @Test: Comprueba que posicionTabla(n) lanza NoSuchElementException si el elemento no existe
108     // y que el mensaje sea "No existe".
109     @Test
110     public void testPosicionNoExiste() {
111         Integer[] datos = { 1, 2, 3 };
112         TablaEnteros objeto = new TablaEnteros(datos);
113
114         NoSuchElementException excepcion = assertThrows(expectedThrowable: NoSuchElementException.class, () -> {
115             objeto.posicionTabla(n: 99);
116         });
```

```java
        String esperado = "No existe";
        assertEquals(esperado, excepcion.getMessage());
    }

    // @Test: Comprueba que el constructor lanza IllegalArgumentException si se le pasa null
    // y que el mensaje sea "No hay elementos".
    @Test
    public void testConstructorErrorNulo() {
        IllegalArgumentException excepcion = assertThrows(expectedThrowable: IllegalArgumentException.class, () -> {
            new TablaEnteros(tabla: null);
        });

        String esperado = "No hay elementos";
        assertEquals(esperado, excepcion.getMessage());
    }

    // @Test: Comprueba que el constructor lanza IllegalArgumentException si se le pasa un array vacío
    // y que el mensaje sea "No hay elementos".
    @Test
    public void testConstructorErrorVacio() {
        Integer[] tablaVacia = new Integer[0];

        IllegalArgumentException excepcion = assertThrows(expectedThrowable: IllegalArgumentException.class, () -> {
            new TablaEnteros(tablaVacia);
        });

        String esperado = "No hay elementos";
        assertEquals(esperado, excepcion.getMessage());
    }
}
```

Hemos chequeado lo siguiente:
De manera positiva (casos correctos)
* new TablaEnteros({1,2,3}) -> se espera que cree el objeto (no null)

* sumaTabla() con {1,2,3,4,5} -> se espera 15
* sumaTabla() con {-1,-2,-3,-4,-5} -> se espera -15
* sumaTabla() con {0,0,0,0,0,0} -> se espera 0

* mayorTabla() con {1,40,20,5,10} -> se espera 40
* mayorTabla() con {-10,-5,-20,-100} -> se espera -5
* mayorTabla() con {7} -> se espera 7

* posicionTabla(30) con {10,20,30,40} -> se espera 2
* posicionTabla(5) con {5,10,15} -> se espera 0

De manera negativa / error (errores controlados)
X new TablaEnteros(null) -> se espera IllegalArgumentException con mensaje "No hay elementos"
X new TablaEnteros(new Integer[0]) -> se espera IllegalArgumentException con mensaje "No hay elementos"
X posicionTabla(99) con {1,2,3} -> se espera NoSuchElementException con mensaje "No existe"